本科毕业论文（设计）

RISC-V 指令集乱序处理器设计

Design of RISC-V Out-of-Order Processor

王靳

哈尔滨工业大学

2026 年 5 月

□毕业论文　□毕业设计　　密级：公开

本科毕业论文（设计）

RISC-V 指令集乱序处理器设计

本　　科　　生：王靳

学　　号：220111012

指　导　教　师：薛睿高级实验师

专　　业：计算机科学与技术

学　　院：深圳校区计算机科学与技术学院

答　辩　日　期：2026 年 5 月

学　　校：哈尔滨工业大学

摘　要

随着开源指令集架构 RISC-V 的快速发展，基于开源生态开展高性能处理器研究已成为体系结构领域的重要方向。乱序执行通过动态调度与寄存器重命名提升指令级并行性，是现代处理器获得高性能的关键机制。本文围绕“RISC-V 指令集乱序处理器设计”，完成了从核心微架构到系统软件支撑的整体实现与验证。

在微架构设计方面，本文实现了前后端解耦的单发射乱序处理器，采用统一物理寄存器的寄存器重命名方案，支持乱序执行与顺序提交，并通过重排序机制实现精确异常；前端分支预测采用 GShare+IJTC+RAS 组合方案。存储层次方面，实现了基于 VIPT 结构且采用 PLRU 替换策略的 ICache 与 DCache，以降低访存开销并兼顾地址转换场景下的访问效率。体系结构支持方面，实现了 M/S/U 三级特权架构、Sv39 虚拟内存机制与中断处理机制，为操作系统运行提供了完整硬件基础。

为保证设计正确性，本文基于 Chisel、Verilator 与 C++20 构建验证平台，结合差分测试与调试追踪工具对关键模块进行系统验证。实验结果表明，所实现处理器在乱序执行、缓存访问、异常中断处理与虚拟内存管理等关键功能上均满足设计预期，并已成功运行 xv6 教学操作系统。本文工作为 RISC-V 乱序处理器的教学与工程实践提供了可复现的实现路径与系统级参考。

关键词： RISC-V；乱序执行；Chisel；SoC；差分测试

Abstract

TODO:

Keywords: RISC-V, Out-of-Order Execution, VIPT Cache, Sv39, xv6

目　录

摘　要 I
Abstract II
第 1 章　绪　论 1
1.1　研究背景与意义 1
1.2　乱序处理器及其相关理论的发展概况 1
1.2.1　乱序执行理论框架的形成与完善 1
1.2.2　精确异常的实现路径 1
1.2.3　寄存器重命名机制与 WAW/WAR 消解 1
1.2.4　分支预测与前端指令供应能力提升 2
1.2.5　开源 RISC-V 与 Chisel 驱动下的新发展 2
1.3　本文的主要研究内容 2
第 2 章　SoC 系统设计思路 3
2.1　设计目标与总体原则 3
2.2　分层总线架构设计 3
2.3　多层次存储与启动链路设计 4
2.4　外设集成与可视化验证思路 5
2.5　本章小结 5
第 3 章　局部多孔质静压轴承的试验研究 6
3.1　引言 6
3.2　多孔质石墨渗透率测试试验 6
3.3　本章小结 6
第 4 章　其他 Typst 使用示例 7
4.1　图表 7
4.2　伪代码 7
4.3　代码块 8
结　论 9
参考文献 10
攻读学士学位期间取得创新性成果 11
原创性声明和使用权限 12
致　谢 13

第 1 章　绪　论

1.1　研究背景与意义

RISC-V 作为开源、模块化、可扩展的指令集架构，正在从学术研究走向产业化部署。相较于传统封闭指令集，RISC-V 在授权成本、生态开放性和可定制能力方面具有显著优势，为高校和科研机构开展处理器全栈设计提供了可行路径。与此同时，面向高性能场景的处理器普遍依赖乱序执行技术来提升指令级并行性和流水线吞吐率，因此围绕 RISC-V 乱序处理器开展研究具有明确的工程价值与教学价值。

从产业需求看，掌握高性能处理器核心微架构的自主设计能力，是推进自主可控芯片技术体系建设的重要基础。从教学与科研实践看，当前公开资料中存在“简单顺序核容易入门、工业级乱序核复杂度过高”的鸿沟，不利于学习者系统理解乱序执行关键机制。基于此，本文以可实现、可验证、可复现为导向，设计并实现一款面向教学与工程过渡场景的 RISC-V 乱序处理器与配套 SoC 平台。

1.2　乱序处理器及其相关理论的发展概况

1.2.1　乱序执行理论框架的形成与完善

20 世纪 90 年代是现代乱序处理器理论体系系统化的关键时期。Smith 与 Sohi 在 1995 年的综述中指出，高性能超标量处理器的核心任务是在保持顺序语义可见性的前提下，将程序中的潜在并行性转化为硬件可执行的并行操作。该综述将处理流程归纳为取指与分支处理、寄存器相关性分析、动态发射与执行、访存数据通信、按程序顺序提交与精确中断支持等关键阶段，为后续乱序微架构设计提供了统一分析框架。

1.2.2　精确异常的实现路径

为在乱序执行条件下保持体系结构可见状态的顺序一致性，精确异常机制是核心设计点。典型实现路径包括：基于 ROB 的顺序提交方案（在提交点统一更新架构状态）；基于 walk-back 的恢复方案（异常时按记录回退未提交状态）；基于 checkpoint 的快速恢复方案（保存关键映射快照并在错误路径上快速回滚）。三类方案分别在硬件开销、恢复时延与实现复杂度之间进行权衡。

1.2.3　寄存器重命名机制与 WAW/WAR 消解

为消除 WAW/WAR 等伪相关并释放指令级并行性，重命名机制形成了多条技术路线。第一类是 ROB-based 方案，结果先写入 ROB 项并在提交时回写架构寄存器；第二类是 Arch Register File + Future File 方案，将提交态与推测态分离管理；第三类是统一物理寄存器文件（PRF）方案，通过 RAT 维护映射关系并在提交后回收旧物理寄存器。统一 PRF 方案因扩展性与实现弹性较好，在现代乱序处理器中应用广泛。

1.2.4 分支预测与前端指令供应能力提升

分支预测能力直接决定前端取指带宽的有效利用率。相关技术从静态预测与一/二位饱和计数器，发展到两级自适应预测（如 GShare、GSelect），再到混合预测与锦标赛选择机制。同时，BTB、RAS 与间接跳转目标缓存等结构被联合用于覆盖条件分支、函数返回与间接跳转场景。随着乱序窗口扩大与流水线加深，分支预测准确率与误判恢复代价已成为影响处理器实际吞吐率的关键因素。

1.2.5 开源 RISC-V 与 Chisel 驱动下的新发展

近年来，RISC-V 开源生态显著降低了高性能处理器研究的实现门槛，推动了乱序处理器从“理论可讲”向“工程可复现”转变。以 BOOM、香山等项目为代表的开源实践展示了乱序处理器在现代开源工具链下的系统化实现路径，也为后续研究提供了可参考的架构与验证经验。

同时，Chisel 等硬件构造语言的发展，使复杂微架构与 SoC 集成的参数化设计、模块复用和工程维护能力明显提升。与之配套的差分测试方法（如基于 Spike 的逐指令对比）进一步提高了功能验证效率，形成了“设计—验证—迭代”闭环。上述新发展使得在课程与科研场景中实现具备完整特权架构、缓存与虚拟内存支持的乱序处理器成为现实可行的工程目标。

1.3 本文的主要研究内容

围绕上述理论与工程发展脉络，本文完成了面向 RISC-V 的单发射乱序处理器与 SoC 系统实现，主要成果如下：

1. 完成 SoC 系统设计与集成，建立了处理器核、总线、存储与外设协同工作的系统级平台。

2. 基于 Chisel 完成关键 IP 核实现，包括 SDRAM 控制器与 Flash 控制器（支持 QSPI-QPI 模式切换）。

3. 完成单发射乱序处理器设计，采用统一物理寄存器重命名方案，实现乱序执行、顺序提交与精确异常支持。

4. 完成基于 VIPT 结构且采用 PLRU 替换策略的 ICache 与 DCache 设计与实现。

5. 完成 RISC-V M/S/U 三级特权架构与 Sv39 虚拟内存机制实现，具备操作系统运行所需的体系结构支持能力。

6. 完成 NVBoard（NJU Virtual Board）集成，实现外设可视化联调与系统级交互验证。

7. 完成 xv6 教学操作系统移植与运行验证。

第 2 章　SoC 系统设计思路

2.1　设计目标与总体原则

本课题 SoC 设计的核心目标是为乱序处理器提供可运行操作系统的完整系统环境，并在保证可调试性的前提下兼顾性能与实现复杂度。为此，系统采用“分层总线 + 多层次存储 + 可视化外设联调”的总体思路：高速访存路径优先保障带宽与时延，低速外设路径优先保障协议兼容与工程可维护性，同时通过统一验证框架降低系统集成阶段的调试成本。

在实现策略上，SoC 结构遵循模块化和可替换原则。处理器核、总线互连、存储控制器和外设控制器通过标准接口连接，使得各模块可以独立迭代。该设计既满足当前单发射乱序处理器的系统集成需求，也为后续微架构升级和外设扩展预留了接口空间。

2.2　分层总线架构设计

SoC 采用 AXI4 与 APB 组合的分层互连架构，以匹配不同设备的速率特征。处理器核心通过 AXI4 主接口接入一级 Crossbar，一级 Crossbar 重点服务主存访问关键路径，仅挂接高带宽存储控制器；二级 Crossbar 连接片上存储与桥接模块，承担中速设备访问；低速外设统一通过 AXI4ToAPB 桥进入 APB 域，由 APB Fanout 完成地址分发。

SoC 整体互连架构如 图 2-1 所示。

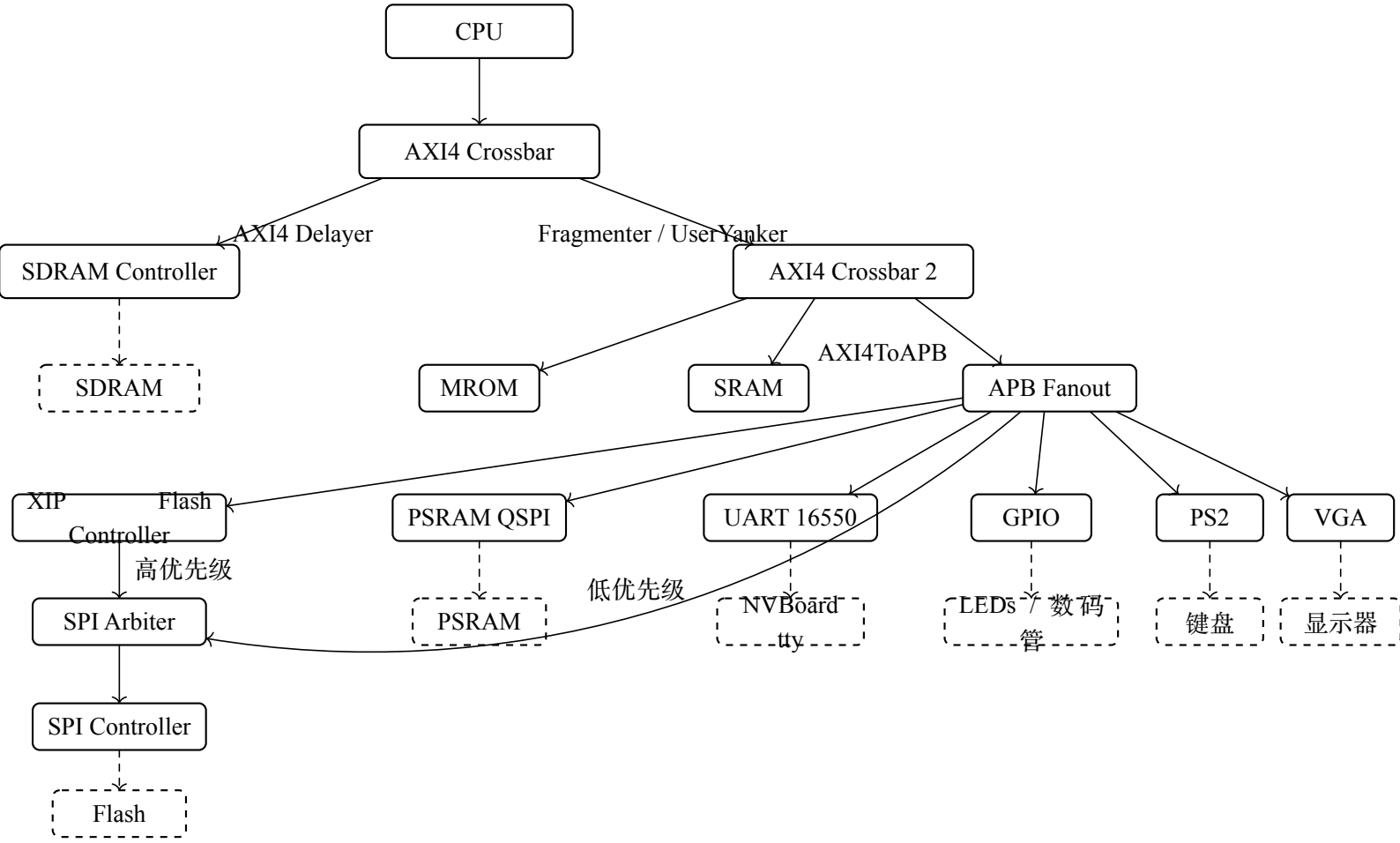

图 2-1　SoC 系统整体互连架构

该分层设计的关键收益在于：其一，高速域与低速域解耦，降低慢设备对关键访存路径的干扰；其二，AXI4 突发与并发事务能力可被主存路径充分利用，而 APB 侧保持实现简单、硬件开销低；其三，桥接与仲裁逻辑集中管理，便于后续扩展新外设或替换控制器实现。

2.3　多层次存储与启动链路设计

存储系统采用 MROM、SRAM、Flash、PSRAM、SDRAM 的多层次组织。MROM 用于放置最小启动代码，SRAM 提供低延迟片上存储，Flash 提供非易失程序镜像并支持 XIP，PSRAM 与 SDRAM 提供更大容量主存。不同存储介质按“启动可靠性—访问延迟—容量”进行功能分工，形成从上电到系统运行的连续存储支撑。

在控制器实现上，关键 IP 核采用 Chisel 实现或重写：SDRAM 控制器面向 AXI4 高带宽路径进行流水化组织，Flash/PSRAM 相关控制器面向 SPI/QSPI/QPI 协议兼容设计。Flash 访问同时支持寄存器方式与 XIP 方式，并通过仲裁机制共享底层 SPI 资源。该设计兼顾了启动阶段可执行性与运行阶段访存效率。

启动链路采用分级引导思路：上电后从 Flash 中的引导入口执行，完成基础初始化后将后续引导或系统镜像加载至 SDRAM，再跳转进入运行态。该流程减少了

直接从串行 Flash 执行大体量程序的性能损失，为操作系统启动提供更稳定的执行环境。

2.4 外设集成与可视化验证思路

外设侧围绕“可用、可测、可观测”进行设计。UART、GPIO、PS2、VGA 等控制器通过 APB 域接入，保证接口一致性与调试便利性。针对教学与开发场景，系统进一步集成 NVBoard，将串口回显、按键输入、数码管显示与 VGA 输出纳入同一仿真闭环，使 SoC 级功能验证从单纯日志比对扩展为可交互的行为验证。

在验证方法上，SoC 级联调与指令级差分测试协同使用：差分测试用于保证体系结构状态正确性，NVBoard 与外设交互用于暴露系统级时序和接口问题，二者互补提升了复杂问题定位效率。基于该思路，系统能够稳定支撑操作系统运行与外设协同工作。

2.5 本章小结

本章从目标约束、总线分层、存储组织、启动链路与外设验证五个方面阐述了 SoC 设计思路。通过 AXI4/APB 分层互连、多层次存储分工、关键控制器 Chisel 化实现以及 NVBoard 可视化联调，系统在性能、可维护性与可验证性之间取得了平衡，为乱序处理器运行 xv6 等操作系统提供了完整平台基础。

第 3 章　局部多孔质静压轴承的试验研究

3.1　引言

在前面几章中，分别对局部多孔质材料的渗透率……

3.2　多孔质石墨渗透率测试试验

……

1 号试样的试验数据见 表 3-1。

表 3-1　1 号试样渗透率测试数据(温度：T=16 ℃ 高度：H=5.31 mm)

供气压力 P_s(MPa)	流量测量 $M'(m^3/h)$	流量修正值 $M(m_3/s)$ $\times 10^{-4}$	压力差 ΔP(Pa)	$\lg \Delta P$	$\lg M$
0.15	0.009	0.023 12	46 900	4.671 17	−5.636 01
0.2	0.021	0.045 84	96 900	4.986 32	−5.338 76

表 3-2　试样渗透率测试数据

供气压力 P_s(MPa)	流量测量 $M'(m^3/h)$	流量修正值 $M(m_3/s)$ $\times 10^{-4}$	压力差 ΔP(Pa)	$\lg \Delta P$	$\lg M$
0.15	0.009	0.023 12	46 900	4.671 17	−5.636 01
0.2	0.021	0.045 84	96 900	4.986 32	−5.338 76

3.3　本章小结

……

第 4 章　其他 Typst 使用示例

4.1　图表

使用 `@fig:` 来引用图片：图 4-1 Lorem ipsum dolor sit.

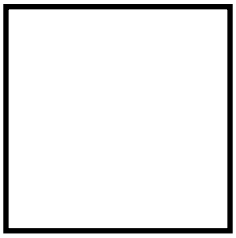

图 4-1　A curious figure.

~~图表之后第一段默认不缩进，如需缩进，可以手动调用 `#indent` 实现缩进。~~

表 4-1　Timing results

t	1	2	3
y	0.3s	0.4s	0.8s

表 4-2　Timing results（三线表）

t	1	2	3
y	0.3s	0.4s	0.8s

使用 `@tbl:` 来引用表格：表 4-1 表 4-2

4.2　伪代码

使用 `@algo:` 来引用伪代码，支持 `algo` 和 `lovelace` 包，如算法 4-1、算法 4-2 和算法 4-3 所示

算法 4-1: 斐波那契数列 1

```
Fib(n):
if n < 0:
    return null
if n = 0 or n = 1:
    return n

let x ← 0
let y ← 1
for i ← 2 to n − 1:   // so dynamic!
    let z ← x + y
    x ← y
    y ← z
```

return $x + y$

算法 4-2: 斐波那契数列 2

$\underline{\text{Fib}}(n)$:

if $n < 0$:
    return null
if $n = 0$ **or** $n = 1$: // you can also
    return n // add comments!
return $\text{Fib}(n-1) + \text{Fib}(n-2)$

算法 4-3: The Euclidean algorithm

input: integers a and b
output: greatest common divisor of a and b

while $a \neq b$ **do**
    if $a > b$ **then**
        $a \leftarrow a - b$
    else
        $b \leftarrow b - a$
    end
end
return a

4.3 代码块

```
fn main() {
    println!("Hello, World!"); // 这是一行注释
}
```

代码 4-1 XXX 代码

与 Markdown 类似，代码可以高亮显示，使用 `@lst:` 来引用代码块： 代码 4-1

结　论

TODO:

参考文献

[1] 林来兴. 空间控制技术[M]. 宇航出版社, 1992.
[2] 杜云祥. 《信息技术与信息服务国际研讨会论文集》 国内作者统计分析[J]. 医学图书馆通讯, 1996, 5(3): 53-54.
[3] 赵耀东. 新时代的工业工程师[Z]. 台北: 天下文化出版社, 1998.
[4] 谌颖. 空间交会控制理论与方法研究[J]. 1992.
[5] Kanamori H. Shaking without quaking[J]. Science, 1998, 279(5359): 2063-2064.
[6] Era G. Plant Physiology: Plant Biology in the Genome Era[J]. SCIENCE-NEW YORK THEN WASHINGTON-, 1998: 331.
[7] 王晓华, 闫其涛, 程智强, 等. 科技论文中文摘要写作要点分析[J]. 编辑学报, 2010(S1): 53-55.
[8] Kopka H, Daly P W, Rahtz S. Guide to LATEX: Vol. 4[M]. Addison-Wesley Boston, MA, 2004.
[9] Test A1, Test A2, Test A3, et al. Test Book: Vol. 1[M]. 2nd ed. Some Place: Some Publisher, 2021.

攻读学士学位期间取得创新性成果

参与的科研项目及获奖情况

哈尔滨工业大学本科毕业论文（设计）

原创性声明和使用权限

本科毕业论文（设计）原创性声明

本人郑重声明：此处所提交的本科毕业论文（设计）《RISC-V 指令集乱序处理器设计》，是本人在导师指导下，在哈尔滨工业大学攻读学士学位期间独立进行研究工作所取得的成果，且毕业论文（设计）中除已标注引用文献的部分外不包含他人完成或已发表的研究成果。对本毕业论文（设计）的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明。本毕业论文（设计）对使用 AI 工具的情况进行了明确标注。

作者签名：Lorem ipsum.　　日期：2026 年 4 月 12 日

本科毕业论文（设计）使用权限

本科毕业论文（设计）是本科生在哈尔滨工业大学攻读学士学位期间完成的成果，知识产权归属哈尔滨工业大学。本科毕业论文（设计）的使用权限如下：

（1）学校可以采用影印、缩印或其他复制手段保存本科生上交的毕业论文（设计），并向有关部门报送本科毕业论文（设计）；（2）根据需要，学校可以将本科毕业论文（设计）部分或全部内容编入有关数据库进行检索和提供相应阅览服务；（3）本科生毕业后发表与此毕业论文（设计）研究成果相关的学术论文和其他成果时，应征得导师同意，且第一署名单位为哈尔滨工业大学。

保密论文在保密期内遵守有关保密规定，解密后适用于此使用权限规定。

本人知悉本科毕业论文（设计）的使用权限，并将遵守有关规定。

作者签名：Lorem ipsum.　　日期：2026 年 4 月 12 日

导师签名：Lorem ipsum dolor.　　日期：2026 年 4 月 12 日

致　谢

TODO: